

BestCommerce: An AI-Powered E-Commerce Platform with Intelligent Business Insights

Abu Md. Ashrafur Rahat Md. Shakib Rafid Shakib Muhammad Farhan Rafid
 Department of Electrical and Computer Engineering, North South University
 CSE299 — Junior Project Design, Section 3, Group 1
 Faculty: Dr. M. Shifat-E-Rabbi (MSRb)

Abstract—This report presents BestCommerce, the final implementation of an AI-powered e-commerce platform developed for CSE299. The system combines a React/Vite storefront with a Django REST Framework backend, PostgreSQL persistence, JSON-based HTTP APIs, JWT authentication, shopping-cart and order workflows, administrative invoice generation, and two locally hosted AI chatbot interfaces using Ollama. The customer chatbot receives a deliberately sanitized product snapshot and answers public shopping questions, whereas the admin chatbot is restricted to authenticated administrators and receives a read-only database summary containing product, stock, order, and sales information. A bidirectional AI firewall normalizes input, filters prompt-injection and protected-data requests, rate-limits chatbot calls, and checks model output for JWTs, bearer tokens, database credentials, secret-key patterns, and protected prompt leakage. The final source contains eight Django applications, eighteen project data models, and fifteen active React page components. This report documents the problem, design, data architecture, implementation, security controls, observable functionality, limitations, and future work strictly from the submitted final source code and project documentation. The project demonstrates how a conventional transactional web system can be combined with local language-model assistance while maintaining a clear separation between public and administrative data contexts.

Index Terms—e-commerce, Django, Django REST Framework, React, PostgreSQL, JWT, Ollama, AI chatbot, prompt injection, API security

I. INTRODUCTION

The project proposal identified two practical problems for small and medium e-commerce operations: customer-facing teams must repeatedly answer product questions, while managers often need to inspect operational data such as inventory and recent orders without manually querying database tables. The same proposal also required conventional storefront functions such as user accounts, product browsing, cart operations, checkout, order records, and administration. The final system implements these concerns in a modular web architecture rather than treating the chatbot as a standalone demonstration.

The resulting application, BestCommerce, uses a browser-based React frontend and a Django backend exposing REST endpoints. PostgreSQL is configured as the active relational database. Authentication is performed

using JSON Web Tokens (JWTs), and protected API requests attach the access token using the standard Bearer header. Product and category endpoints can be read publicly, while carts, orders, notifications, profiles, inventory operations, and administrator AI access are governed by Django REST Framework permission classes. The final project also includes Docker configuration for an isolated Django/PostgreSQL development environment and an administrative billing view that renders an order into a printable PDF invoice.

The primary technical contribution of the final iteration is the dual-chat architecture plus an explicit AI security layer. Rather than granting the language model arbitrary database access, Django constructs bounded textual snapshots. The customer model receives only active product name, category, price, description, and a coarse “Available/Out of stock” state. Exact stock counts and operational records are excluded. The admin model receives a larger read-only snapshot containing counts, low-stock items, a bounded catalog, recent orders, order-status totals, and aggregated sales information. This design keeps database querying in deterministic server-side Python while using the model only for natural-language interpretation and presentation.

A. Objectives and Scope

The final code supports the following project objectives: (1) customer registration and JWT login; (2) category and product browsing with images; (3) authenticated cart creation, update, and deletion; (4) checkout that converts cart lines into persistent order items; (5) order history and order-detail interfaces; (6) staff-side invoice PDF rendering; (7) customer AI product assistance; (8) administrator AI business queries; (9) permissions, CORS configuration, rate limiting, and AI input/output filtering; and (10) Docker-based backend/database execution. The report does not claim functionality that is not completed in the source. In particular, the current checkout records a selected payment method but does not integrate an external payment gateway, stock is not automatically decremented during order creation, the React profile form currently logs submitted data rather than persisting it, and the database-backed human-support models are not connected to the active customer AI page.

II. TECHNICAL FOUNDATION AND RELATED WORK

The system follows widely used web-development patterns rather than introducing a custom framework. Django 4.2 provides URL routing, object-relational mapping (ORM), authentication integration, templates, media handling, and the application structure used by the backend [4]. Django REST Framework (DRF) converts model-backed logic into JSON APIs through serializers, API views, ViewSets, routers, permission classes, and throttling [5]. The frontend uses React components and state hooks, while React Router maps browser paths such as `/products`, `/cart`, and `/orders/:id` to page components [6]. Vite supplies the frontend development/build environment, and Tailwind CSS utility classes are used throughout the JSX for styling.

PostgreSQL is configured in `backend/settings.py` as the active database engine and is also defined as the database service in `docker-compose.yml`. Django models therefore describe relational tables, foreign keys, one-to-one relationships, uniqueness rules, and timestamps, while the ORM translates Python queries into SQL. SimpleJWT provides access and refresh tokens for DRF [7]. In the submitted configuration, access tokens live for 60 minutes and refresh tokens for seven days. CORS is restricted to the two configured local frontend origins, including Vite’s development port 5173.

The AI layer uses the Python Ollama client and the local model identifier `phi3:mini`. Ollama makes it possible for the backend to send structured system/user messages to a local model process rather than embedding a remote commercial API key in the application [9]. The implementation is deliberately retrieval-constrained: Django creates the data context itself, and the system prompts direct the model to answer only from that context. This approach resembles retrieval-augmented application patterns, but the “retrieval” here is deterministic ORM aggregation over the application’s own database rather than vector search.

Traditional e-commerce repositories commonly combine catalog, cart, order, authentication, and administration functionality. The distinguishing feature in this project is the separation between public AI context and private administrative AI context, plus the additional filtering layer before and after model inference. The customer chatbot is not treated as a privileged agent; it cannot directly execute ORM operations or modify carts/orders. The admin chatbot is also read-only: it receives a bounded database snapshot and its system prompt explicitly forbids claims of creating, deleting, or changing database records. This design reduces the authority of language-model output and keeps transactional state changes inside normal Django endpoints.

III. SYSTEM ARCHITECTURE

A. Layered Architecture

The application can be viewed as four cooperating layers: presentation, API/application logic, persistence,

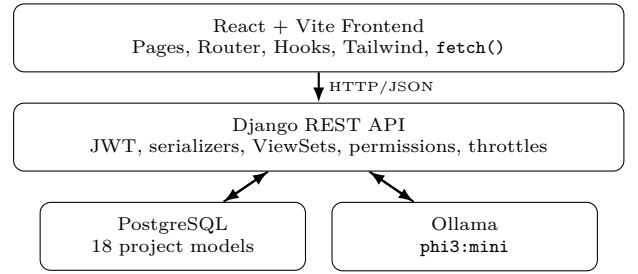


Fig. 1. Final development architecture implemented in the submitted source.

TABLE I
PROJECT MODEL GROUPS

Module	Models and purpose
Accounts	User, Profile: identity and customer/profile information.
Store	Category, Product, ProductImage, Inventory: catalog, media, stock and adjustment history.
Cart	Cart, CartItem: one cart per authenticated user and product-line snapshots.
Orders	Order, OrderItem, Payment, Invoice: checkout records, line items, payment metadata and invoice file records.
Chat	ChatSession, ChatMessage, ChatMessageAttachment, HumanSupportSession: persistent support-session schema.
Core	Setting, Notification: application settings and per-user notifications.

and local AI. Figure 1 summarizes the deployed development structure. React renders pages, stores JWTs in browser local storage, and uses `fetch()` to exchange JSON with Django. Django performs authentication, validation, business logic, ORM queries, and response serialization. PostgreSQL stores persistent records. Ollama is called only by two chatbot endpoints and receives server-generated context.

At backend startup, `manage.py` loads the Django project configuration. `backend/settings.py` registers built-in Django apps, DRF, CORS middleware, and eight internal applications: `accounts`, `cart`, `chat`, `chatbot`, `core`, `orders`, `store`, and `billing`. `backend/urls.py` then mounts JWT routes and each application’s URL configuration under the `/api/` prefix. DRF routers automatically create list/detail routes for most ModelViewSets.

B. Data Model

The final source contains eighteen project model classes. Table I groups them by concern. The custom `User` extends Django’s `AbstractUser`; email is unique, phone is optional, and flags identify guest/customer status. A one-to-one `Profile` stores addresses and an optional profile image. Catalog data uses `Category`, `Product`, `ProductImage`, and `Inventory`. Categories may point to a parent category, allowing hierarchical catalogs. Products use `DecimalField` for price and compare-price values and store a direct integer stock value. `Inventory` acts as an adjustment log rather than the sole stock source.

`Cart` may theoretically represent an authenticated or guest cart, but the active serializer requires authentication when adding items. `CartItem` enforces a unique cart/product pair and records `price_at_add`, preserving the product price at the time the item enters the cart. During checkout, `OrderSerializer.create()` creates an `Order`, generates a 12-character order number from a UUID, copies each cart line to `OrderItem` using the stored price snapshot, and then deletes all cart items. Historical order items use `on_delete=models.PROTECT` for the product relation, protecting order history from accidental product deletion.

The order total is computed instead of stored as a single fixed field. For line items $i = 1, \dots, n$, the subtotal is

$$S = \sum_{i=1}^n q_i p_i, \quad (1)$$

where q_i is quantity and p_i is the recorded order-item price. Shipping is calculated from the free-text address:

$$C_s = \begin{cases} 70, & \text{if the address contains "Dhaka"} \\ 140, & \text{otherwise.} \end{cases} \quad (2)$$

The order property returns $T = S + \text{tax} + C_s - \text{discount}$. The frontend independently presents the same Dhaka/outside-Dhaka shipping rule during checkout, while the backend remains the authoritative source of the serialized order total.

C. API Organization

The active API is modular. Authentication is available at `/api/token/` and `/api/token/refresh/`. Registration is at `/api/accounts/register/`. Catalog routers expose categories, products, and inventory; cart routers expose carts and cart items; order routers expose orders, payments, and invoices; chat routers expose session/message/human-support records; and core routers expose settings and notifications. The AI endpoints are explicitly declared as `/api/ai/chat/` and `/api/ai/admin-chat/`. This organization keeps conventional database CRUD separate from the two language-model endpoints.

DRF serializers define which model fields enter or leave JSON. For example, `ProductSerializer` nests gallery images and inventory logs, while `CartItemSerializer` embeds `product_detail` so the React cart can display name/image/price without a second product lookup. A custom `FlexibleImageField` normalizes media paths and preserves already absolute HTTP, HTTPS, and data URLs. This is important because the frontend’s `getImageUrl()` helper also handles relative media paths and optional `VITE_API_BASE` configuration.

IV. IMPLEMENTATION

A. Authentication, Authorization, and User Flow

Registration is implemented by `RegisterView`, a DRF `CreateAPIView` with `AllowAny`.

`RegisterSerializer.create()` calls Django’s `create_user()`, ensuring password hashing is performed by the authentication framework rather than storing a plaintext password. The serializer sets `is_customer=True`. In the current React form, the email value is used as both `username` and `email`; the first-name and last-name fields are displayed but are not sent to the backend. This allows the login page to label its credential field “Username or Email” while actually sending the entered value as the SimpleJWT `username` field.

After a successful token request, the frontend stores `data.access` and `data.refresh` as `accessToken` and `refreshToken` in `localStorage`. `getAuthHeaders()` returns an `Authorization: Bearer <token>` header, and `getJsonHeaders()` adds `Content-Type: application/json`. Protected `ViewSet`s combine `IsAuthenticated` with custom ownership logic. `IsOwnerOrStaff` allows staff/support users while checking direct user ownership, numeric user IDs, cart ownership, or chat-session ownership for normal users. Inventory access uses the support/staff permission class. Admin chatbot access uses DRF’s `IsAdminUser`.

The security design is appropriate for a semester development environment but not yet production-hardened. The source retains `DEBUG=True`, a hard-coded Django secret key, and a fallback database password. Although database connection values can be supplied from environment variables, production deployment should remove secret fallbacks, disable debug mode, set explicit allowed hosts, enable HTTPS and secure cookie/header settings, and apply a stronger browser token-storage strategy.

B. Product Catalog and Inventory

`CategoryViewSet` and `ProductViewSet` permit unauthenticated reads and require authentication for modifications through `IsAuthenticatedOrReadOnly`. Product queries filter `is_active=True`; an optional `category` query parameter narrows results by category ID. The React Products page loads `/api/store/products/` in a `useEffect()` hook, stores the returned JSON in component state, filters it with a search input, and posts selected product IDs to the cart-item endpoint. `ProductDetail` similarly fetches a single product by URL parameter and can add it to the cart.

The inventory schema separates the current stock integer on `Product` from `Inventory` adjustment-log records. Each log stores a positive or negative quantity change, reason, note, optional creator, and timestamp. This provides a basis for audit history, although the current `InventoryViewSet.perform_create()` should be reviewed before production because it passes the user object to a `created_by_id` keyword rather than saving it through the relation field directly. More importantly, order creation currently does not reduce `Product.stock`; therefore the current stock field must be maintained separately through administration or inventory logic.

C. Cart, Checkout, Orders, and Billing

Adding a product uses `CartItemSerializer.create()`. The serializer obtains the request user from `serializer.context`, creates the user's cart if needed, then calls `get_or_create()` for the cart/product pair. A new line receives the product's current price as `price_at_add`; an existing line increases quantity. Updating a line with quantity zero or below deletes it. The React cart page fetches only the authenticated user's cart items, provides plus/minus quantity operations through update requests, and deletes rows through the detail endpoint.

Checkout gathers shipping address, billing address, and a selected payment-method string. The UI offers credit card, PayPal, and bank transfer labels, but these values are metadata choices only: no external processor, card collection, PayPal authorization, bank API, transaction verification, or webhook flow is present. The order is created with `payment_status='pending'`. The backend copies cart lines into order lines and clears the cart, so the project implements order placement but not settlement.

Order history and detail pages call the protected orders endpoint with Bearer authentication. `OrderViewSet.get_queryset()` returns all orders to staff/support users and only owner orders to ordinary users. The associated `PaymentViewSet` and `InvoiceViewSet` apply the same scope. Independently, the billing application includes `admin_invoice_pdf_view()`, protected by Django's `staff_member_required` decorator. It fetches any order by ID, renders `billing/templates/invoices/bill.html`, and passes the rendered HTML to xhtml2pdf's `pisa.CreatePDF()` [10]. The response is displayed inline with a filename derived from the order number.

D. React Frontend

The active frontend resides in `my-app/`; `my-app-OLD/` is a legacy copy and is not the active implementation discussed here. `App.jsx` creates a `BrowserRouter`, persistent navigation bar, and fifteen page routes: home, login, register, products, product detail, cart, checkout, order history, order detail, categories, category detail, profile, customer chat, admin chat, and a catch-all not-found page. React's `useState()` stores form values, remote data, errors, messages, and loading states. `useEffect()` performs initial data loading for products, categories, carts, and orders.

JSON is the main data interchange format. POST/PUT/PATCH requests call `JSON.stringify()` on JavaScript objects, Django/DRF parses request bodies into serializer data, serializers validate and map fields to models, and DRF returns JSON responses. React calls `response.json()` and updates state. This cycle separates UI rendering from backend persistence and allows the same API to be used by other clients in the future.

E. Dual AI Chatbot Design

The customer chatbot and admin chatbot share the Ollama client but differ in authentication, context, system prompt, and rate limit. `CustomerChatView` uses `AllowAny`, disables authentication classes for the endpoint, applies a scoped throttle of 10 requests per minute, and receives only one validated `message` field. `build_customer_product_snapshot()` queries up to 30 active products, joins categories, and converts stock to only "Available" or "Out of stock". Exact stock numbers, orders, users, and sales values are not included. The system prompt instructs the model to answer only from that public catalogue, avoid invention, and never claim that it placed an order or added an item to a cart.

`AdminChatView` requires `IsAdminUser` and uses a 20-request-per-minute scope. Its snapshot function counts all/active products, categories, users, marked customers, total orders, and today's orders. It also calculates order-status counts, selects up to ten low-stock products with stock less than or equal to 10, includes up to 20 catalog products, lists up to five recent orders, and aggregates up to five top-selling products using `Sum(quantity)` and an ORM expression for quantity multiplied by price. Recorded sales value is aggregated only for orders whose status is paid, shipped, or delivered. The model receives the snapshot as text and is explicitly read-only.

This design avoids connecting arbitrary natural-language model output directly to SQL or update operations. All ORM queries are prewritten Python. Consequently, even if the model misunderstands an admin question, it only summarizes the supplied snapshot; it has no tool that can delete a user, alter stock, create an order, or execute a raw query.

F. Bidirectional AI Firewall

The file `chatbot/ai/_firewall.py` adds deterministic checks around model inference. Figure 2 shows the sequence. Before any pattern matching, `normalize_input()` applies Unicode NFKC normalization, removes zero-width characters, removes unwanted control characters, collapses repeated whitespace, and trims the string. NFKC reduces visually or semantically equivalent Unicode forms to a more consistent representation, making simple security patterns harder to evade with compatibility characters. Zero-width and control-character stripping removes characters that may split attack phrases without being obvious to a human reviewer.

General input regular expressions reject requests to ignore prior/system/developer instructions, reveal hidden/system prompts, perform jailbreak/prompt-injection bypasses, or expose credentials such as passwords, keys, access tokens, refresh tokens, or JWTs. In customer mode, additional patterns block attempts to enter admin mode, request database snapshots/all users/customer emails, request order/revenue information, or obtain exact inventory

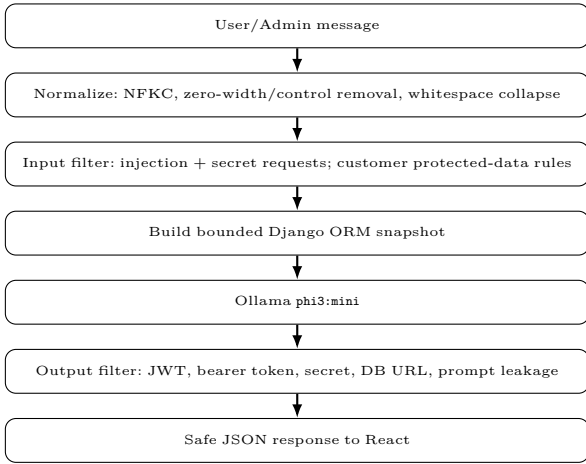


Fig. 2. Bidirectional chatbot security pipeline.

counts. Blocked requests return a safe predefined reply and identify the firewall layer/reason in JSON.

After Ollama responds, `check_output()` scans the reply before it reaches React. It rejects strings matching a JWT-like three-segment token beginning with `eyJ`; secret-looking key/value assignments; **Authorization: Bearer** content; credential-bearing PostgreSQL/MySQL/MongoDB URLs; Django insecure secret-key patterns; and exact markers derived from the internal system prompts. This output stage matters because prompt filtering alone cannot guarantee that a language model never emits sensitive-looking content. The implementation therefore treats model output as untrusted as well as model input.

Rate limiting is configured separately in DRF settings and attached through `ScopedRateThrottle`. The customer and admin endpoints use different scopes and limits. The combination of role permissions, minimal context, input filtering, throttling, output filtering, and read-only database snapshots provides defense in depth rather than relying on a single system prompt.

G. Deployment Configuration

The Dockerfile starts from `python:3.12-slim`, sets an application working directory, installs `requirements.txt`, copies the project, exposes port 8000, and starts the Django development server. `docker-compose.yml` defines a PostgreSQL 16 Alpine service and a Django service. The Django service depends on the database, mounts the source directory, injects database environment variables, runs migrations, and then starts the server. A named Docker volume preserves PostgreSQL data. The React application remains a separate Vite development process in the final repository; it is not built into the provided Docker Compose stack.

The dependency file records Django 4.2.30, DRF 3.16.1, SimpleJWT 5.5.1, django-cors-headers, Pillow for image fields, psycopg2-binary for PostgreSQL, Ollama 0.6.2, and

TABLE II
FEATURE-TO-FILE TRACEABILITY

Feature	Principal implementation files
Register/login	<code>accounts/serializers.py</code> , <code>accounts/views.py</code> , <code>backend/urls.py</code> , <code>Register.jsx</code> , <code>Login.jsx</code> .
Catalog	<code>store/models.py</code> , <code>store/serializers.py</code> , <code>store/views.py</code> , <code>ProductDetail.jsx</code> , <code>Products.jsx</code> .
Cart	<code>cart/models.py</code> , <code>cart/serializers.py</code> , <code>cart/views.py</code> , <code>Cart.jsx</code> .
Checkout/orders	<code>orders/models.py</code> , <code>orders/serializers.py</code> , <code>orders/views.py</code> , <code>Checkout.jsx</code> , <code>OrderHistory.jsx</code> .
Billing	<code>billing/views.py</code> and the <code>invoices/bill.html</code> template.
Customer AI	<code>chatbot/views.py</code> , <code>chatbot/ai_firewall.py</code> , <code>Chat.jsx</code> .
Admin AI	<code>chatbot/views.py</code> , <code>chatbot/ai_firewall.py</code> , <code>AdminChat.jsx</code> .

`xhtml2pdf` 0.2.17. The active frontend package file records React 19.2.7, React Router 7.18.1, Tailwind CSS 4.3.3, and Vite 8.1.1. These explicit versions improve reproducibility within the submitted development snapshot.

V. MODULE TRACEABILITY AND INTERNAL INTERFACES

A useful way to understand the final system is to trace each user-visible capability to a small set of files rather than treating the repository as one monolithic program. The backend project package, `backend/`, contains global settings, root URL dispatch, ASGI/WSGI entry points, and reusable permissions. Each business domain then owns its models, serializers, URLs, views, migrations, and administration registration. This organization follows Django’s application boundary: data rules stay close to the domain that owns them, while `backend/urls.py` only connects the domains together. The React frontend follows a similar separation. `App.jsx` defines navigation-level routing, `apiConfig.js` centralizes API-base and authentication-header behavior, `Navbar.jsx` provides shared navigation, and page components own their local forms and request logic.

Table II maps the principal user flows to the files that implement them. The mapping also shows that most full-stack features have three stages: React initiates the request, DRF validates and authorizes it, and a model/serializer performs persistence or calculation. The AI endpoints add a fourth stage because the Django view builds context and invokes Ollama before returning JSON.

The serializers are particularly important internal interfaces. A Django model describes storage, but a serializer determines the API representation and validation boundary. This prevents the frontend from writing arbitrary database columns simply because they exist in a model. For example, order ownership and generated order numbers are read-only in `OrderSerializer`; the client supplies addresses and payment-method metadata, while the serializer determines the authenticated user and creates the order number. Likewise, `CartItemSerializer` does not allow

the browser to choose `price_at_add`; it reads the current product price server-side when the line is first created. This is a basic integrity rule because trusting a client-supplied historical price would allow a modified request to alter checkout value.

Ownership is enforced at both query and object-permission levels. Cart and cart-item queriesets are filtered to the request user. Order, payment, and invoice ViewSets return owner-scoped records to ordinary accounts but broaden the queryset for staff/support users. Notifications are filtered to the target user. These patterns reduce accidental information disclosure even when a client manually changes a numeric object ID in a URL. The custom `IsOwnerOrStaff` permission then provides an additional object-level check for several common relationship shapes.

The media path is another cross-layer interface. Django stores images through `ImageField` values and serves media under `/media/` while debugging. The custom product/category image serializer converts stored file names into media-style paths unless an image is already represented as an absolute web/data URL. React’s `getImageUrl()` then combines those paths with the optional API base. As a result, the same JSX can render local backend media whether the frontend and backend share an origin or run on separate development ports.

Repository configuration also exposes an important distinction between active and retained artifacts. `my-app/` is the current React client referenced by the final code; `my-app-OLD/` preserves an earlier frontend state and should not be described as a second running client. Similarly, `db.sqlite3` exists in the archive, but `backend/settings.py` selects PostgreSQL as the configured default engine. The correct explanation in a viva is therefore that SQLite is a retained file, whereas the active backend configuration expects PostgreSQL. Docker Compose reinforces this by provisioning a PostgreSQL service and passing its hostname/credentials to Django through environment variables.

These traceability relationships are useful for maintenance as well as presentation. When a cart quantity bug occurs, the relevant path is React `Cart.jsx` → cart-item API route → `CartItemViewSet` → `CartItemSerializer.update()` → `CartItem`. When an AI security issue occurs, the path is React chat page → chatbot API view → normalization/input check → bounded snapshot → Ollama → output check → JSON response. Such deterministic paths make it possible to explain and debug the system feature by feature.

VI. FUNCTIONAL VERIFICATION AND RESULTS

The course emphasizes code demonstration and viva, so this section distinguishes features that can be inferred directly from the implemented code from claims that would require a separate benchmark. The repository’s individual Django `tests.py` files are largely scaffolds and no comprehensive automated integration test suite

TABLE III
IMPLEMENTATION STATUS FROM FINAL SOURCE

Area	Final-code status
Registration/login	Implemented with Django user creation and SimpleJWT access/refresh tokens.
Catalog	Public active categories/products with media and category filtering.
Cart	Authenticated add, increment, set quantity, delete and owner scoping.
Checkout/orders	Cart copied to order items, UUID-derived order number, cart cleared.
Payments	Metadata models/fields exist; no external payment processor.
Invoices	Staff-only HTML-to-PDF order rendering is implemented.
Customer AI	Public product-context chat through Ollama with 10/min throttle.
Admin AI	Admin-only read-only business snapshot chat with 20/min throttle.
AI firewall	Input normalization/filtering plus output secret/token/prompt checks.
Profile page	Backend model/API exists; active React form currently only logs submit data.
Human takeover	Database models/ViewSets exist; not wired into the active AI chat UI.
Automated tests	App test modules exist but comprehensive test coverage is not present.

is included. Accordingly, this report does not invent pass percentages, latency numbers, model-accuracy scores, or security-detection rates. Instead, Table III summarizes the concrete implementation state visible in the final source.

A. End-to-End Request Paths

A normal customer flow begins at the React registration page. The form POSTs JSON to the registration endpoint, Django validates it, hashes the password through `create_user()`, and returns a user representation. Login POSTs credentials to the token endpoint. Once tokens are stored, the browser can request cart/order resources with the Bearer access token. Product reading does not require a token. Adding a product posts `product` and `quantity`; the serializer creates/reuses a cart and a cart-item row. Checkout posts address/payment-method metadata to the order ViewSet, whose serializer reads the same user’s cart, creates the order and order-item snapshots, and removes cart items. The order history page then retrieves only orders visible to the authenticated user.

The customer AI flow is separate from checkout. The browser sends a single question to `/api/ai/chat/`; the backend validates and normalizes it, applies customer-mode input rules, constructs the public product snapshot, invokes Ollama, filters the resulting text, and returns JSON with `reply` and `blocked`. Thus the chatbot can recommend products but cannot perform purchases. The admin flow follows the same security pipeline but additionally requires a staff/admin user and supplies the database summary instead of the public catalog.

B. Security Properties Demonstrated by Structure

The source enforces several concrete trust boundaries. First, Django serializers validate request data before model

writes or model inference. Second, ordinary cart and order queries are filtered by the authenticated user; staff/support users receive broader visibility only where the `ViewSet` explicitly permits it. Third, the customer chatbot is denied privileged database context by construction, not merely by a natural-language warning. Fourth, administrator chatbot requests require `IsAdminUser`. Fifth, the model receives data through generated snapshots and has no update tool. Sixth, suspicious user messages can be rejected before inference, and suspicious model output can be replaced before delivery. Seventh, throttles reduce high-frequency chatbot requests.

The output firewall’s JWT pattern is especially relevant to authentication data. A typical JWT contains three Base64URL-like segments separated by dots, and encoded JSON headers often begin with `eyJ`. The regular expression looks for this shape rather than decoding the token. This is a leakage detector, not a JWT validator: its purpose is to stop token-looking content from appearing in chatbot output. Bearer-header and secret-assignment checks provide additional coverage for credentials that do not match the JWT expression.

VII. LIMITATIONS AND RISK ANALYSIS

The final project achieves the main semester objectives, but several limitations are visible in the source and should guide interpretation of the demonstration. First, development secrets and debug settings remain in the configuration. They are acceptable only for local development and must be replaced by environment-only secrets, production host settings, and HTTPS before public deployment. Second, refresh tokens are stored in browser local storage and rotation is enabled without a complete frontend refresh/expiry workflow. A hardened deployment would define automatic refresh handling, logout/revocation behavior, and a storage model chosen against the application’s XSS/CSRF threat model.

Third, checkout is not a real payment system. The displayed payment-method choices are strings sent to Django and recorded on the order. The source does not collect card details or integrate a payment provider. The related `Payment` model provides a future persistence shape but is not automatically created by `OrderSerializer.create()`. Fourth, the current order path does not validate cart quantity against stock or decrement stock after ordering, creating a possible mismatch between displayed availability and actual fulfillment capacity.

Fifth, the profile feature is split across backend and frontend development stages. The backend contains a profile model, serializer, router, and ownership permission, but the active React `Profile.jsx` only prints submitted form state to the browser console. Sixth, the `chat` application contains a structured schema for sessions, messages, attachments, human takeover, and support-agent access, yet the active `Chat.jsx` directly calls the stateless AI

endpoint and does not create those records or trigger human takeover. Therefore human-support persistence is a designed backend capability rather than a completed end-to-end feature.

Seventh, the AI firewall is intentionally lightweight and regular-expression based. Normalization helps against simple Unicode obfuscation, but regex rules cannot prove semantic safety. Attack wording can be paraphrased, and benign text can resemble a blocked pattern. The firewall should therefore be treated as one defensive layer. The stronger architectural controls are the bounded snapshots, role permissions, lack of model write tools, and output filtering. Finally, the project lacks a comprehensive automated test suite. This limits regression confidence and makes manual demonstration more important.

VIII. FUTURE WORK

The highest-priority future work is completing transactional integrity. Order placement should use database transactions, validate requested quantity against stock, lock or safely update stock, create an inventory ledger entry, and prevent overselling under concurrent requests. A real payment provider should replace the placeholder payment-method metadata. Provider callbacks/webhooks should update `Payment.status` and `Order.payment_status` only after verified settlement. Invoice generation can then be tied to paid/completed states, with persistent generated files or regenerated immutable views as required.

The profile page should be connected to the existing profile API, including retrieval, create/update behavior, authenticated ownership checks, and image upload. Authentication should add frontend refresh-token handling, explicit logout, expiry recovery, and production security configuration. Secrets must leave source control. Docker Compose can be expanded to build and serve the React production bundle behind a reverse proxy, while Ollama service configuration can also be containerized or documented as an external dependency.

The human-support models provide a natural next step for the chatbot. The customer AI endpoint could create a `ChatSession`, persist user/bot messages, and flag `is_human_takeover` based on an explicit user request or escalation condition. A staff dashboard could claim sessions, attach a human agent, and complete `HumanSupportSession` records. This would connect the current database design to the originally proposed disjointed-support problem.

For the AI layer, future security work should add structured security tests covering prompt-injection paraphrases, Unicode edge cases, secret-shaped outputs, permission failures, and throttle behavior. The model context can be converted from long free-form snapshots to structured, minimal JSON-like facts and bounded query intents. The admin assistant could expose a small allow-listed tool layer for predefined analytical queries, but any state-changing

tool should require explicit authorization and server-side validation rather than trusting model-generated actions.

Finally, software-quality work should add unit tests for model properties and serializers, API tests for permissions and ownership, integration tests for cart-to-order conversion, and frontend tests for authentication/error states. Continuous integration can run these checks on each branch update. Performance testing can then measure API latency, database query count, and chatbot response time using actual repeatable workloads rather than anecdotal observations.

IX. CONCLUSION

BestCommerce integrates a conventional e-commerce web application with two differently privileged local AI assistants. The final implementation uses Django/DRF to define persistent models, REST endpoints, permissions, serialization, order logic, billing, and model-context construction; React/Vite implements the browser interface; PostgreSQL supplies relational persistence; JWT secures authenticated requests; and Ollama provides local natural-language generation. The system’s most important architectural decision is that the language model does not receive unrestricted database access. The customer assistant sees a sanitized product catalogue, while the administrator assistant sees a server-generated read-only business snapshot. A bidirectional firewall, rate limiting, role checks, and output leakage detection add further controls.

The source also shows the boundaries of the current prototype: payment is not externally processed, stock is not automatically decremented at checkout, profile persistence is not connected in the active UI, human takeover is modeled but not integrated into the direct chatbot page, and automated test coverage is limited. Documenting these

limitations is necessary for an accurate final report and provides a clear roadmap for future engineering. Within the CSE299 scope, the project demonstrates a modular full-stack architecture and a practical method of adding AI assistance without giving a language model direct authority over transactional data.

ACKNOWLEDGMENT AND TOOL DISCLOSURE

The team used publicly available framework documentation, local/open-source development tools, Git/GitHub for version control, and AI-assisted debugging/documentation during development. ChatGPT was used as an assistance tool; technical statements in this final report were checked against the submitted final source code and project documents rather than treated as authoritative model output.

REFERENCES

- [1] M. S.-E.-Rabbi, “Course Manual: CSE 499A/B, CSE 299,” course document, North South University, 2026.
- [2] CSE299 Section 3 Group 1, “Project Proposal: AI-Powered E-Commerce Platform with Intelligent Business Insights,” project document, 2026.
- [3] CSE299 Section 3 Group 1, “BestCommerce Final Source Code (Master.zip),” submitted project source, 2026.
- [4] Django Software Foundation, “Django Documentation, version 4.2,” <https://docs.djangoproject.com/en/4.2/>.
- [5] Encode OSS Ltd., “Django REST Framework Documentation,” <https://www.django-rest-framework.org/>.
- [6] Meta Platforms, Inc., “React Documentation,” <https://react.dev/>.
- [7] Jazzband, “Simple JWT Documentation,” <https://django-rest-framework-simplejwt.readthedocs.io/>.
- [8] PostgreSQL Global Development Group, “PostgreSQL Documentation,” <https://www.postgresql.org/docs/>.
- [9] Ollama, “Ollama Documentation,” <https://docs.ollama.com/>.
- [10] xhtml2pdf Project, “xhtml2pdf Documentation,” <https://xhtml2pdf.readthedocs.io/>.
- [11] Docker, Inc., “Docker Documentation,” <https://docs.docker.com/>.
- [12] OpenAI, “ChatGPT,” AI assistance tool, accessed 2026.